

PID CONTROLLER

Name: Purva Kaushalbhai Shah

Group members name:

Ashwini Bukke

Anushri Sanodia

Siddharth Verma

Table of Contents:

Objective

Theory and Background

Known parameters

Formulae

Readings

Results and Discussion

Conclusion

References

Objective:

The objective of this experiment was to study the feedback control of temperature using a heater input and analyze the performance of different controllers : Proportional (P), Proportional-Integral (PI), and Proportional-Integral-Derivative (PID). The experiment was conducted in three parts to understand the system dynamics, determine the appropriate controller parameters, and evaluate the effectiveness of control strategies in minimizing error and achieving a stable response.

The experiment consisted of three main phases:

1. System Identification Without a Controller

In the first phase, Scilab and a heater-fan module were used to study the system's behavior without any control. A step input was applied to the heater, and the temperature response was recorded over time using a thermocouple. Based on this data, the system's transfer function was determined to model its dynamic characteristics.

2. Controller Design and Simulation

Using the Cohen-Coon tuning method, the controller parameters (K_c , T_i , and T_d) were calculated for P, PI, and PID controllers. These controllers were then simulated in MATLAB & Simulink, allowing observation of the system's response through scope plots. This step provided insights into how different control strategies affect system behavior.

3. Implementation and Performance Analysis

The tuned PID controller parameters were implemented in Scilab, and the experiment was repeated with the heater-fan module. The error between the heater's input and the thermocouple's output temperature was analyzed to assess the controller's effectiveness in minimizing deviations. The results highlighted how different controllers influenced temperature stability, overshoot, and steady-state error.

Theory and Background:

Control systems are essential in engineering for maintaining the stability and performance of dynamic systems. They regulate variables like temperature, pressure, speed, and flow to achieve desired outcomes. A controller is a key component in a control system, responsible for adjusting inputs to maintain the desired output despite disturbances.

Feedback is crucial in control systems because it continuously monitors the system's output and adjusts the input accordingly. This ensures stability, error reduction –(minimized deviations from the desired setpoint), disturbance rejection and improved performance (faster and more

accurate responses). Without feedback, a system operates in an open-loop manner, meaning it cannot correct errors or adapt to disturbances. In contrast, closed-loop systems with feedback adjust their behavior dynamically, making them essential for precise and reliable control applications.

In temperature regulation, feedback control is a widely used method, wherein the system continuously compares the actual temperature with the desired setpoint and applies corrective actions to minimize the difference. The controllers used are:

1. **Proportional (P) Controller:** It applies a correction proportional to the error (difference between desired and actual output). It improves response speed but may result in steady-state error.
2. **Proportional-Integral (PI) Controller:** Combines proportional action with an integral term to eliminate steady-state error by accumulating past errors. It is effective for systems requiring accuracy without excessive oscillations.
3. **Proportional-Integral-Derivative (PID) Controller:** Incorporates proportional, integral, and derivative terms. The derivative term predicts system behavior to improve stability and reduce overshoot, making PID controllers highly effective for achieving precise and stable control.

A widely implemented feedback control strategy is the Proportional-Integral-Derivative (PID) controller, which fine-tunes the system response by combining three control actions:

1. **Proportional (P) Control** – Reacts to the current error by applying a correction proportional to the difference between the measured temperature and the setpoint. It ensures a quick response but often leaves a steady-state error.

$$u(t) = K_c e(t)$$

2. **Integral (I) Control** – Eliminates steady-state error by integrating past errors over time. However, excessive integral action can cause overshoot and instability if not properly tuned.

$$u(t) = K_c e(t) + K_I \int e(t) dt, K_I \text{ is the integral gain}$$

3. **Derivative (D) Control** – Predicts future error trends by analyzing the rate of change of the error signal. It helps reduce overshoot and oscillations, improving overall stability.

$$u(t) = K_c e(t) + K_D \frac{d e(t)}{dt}, K_D \text{ is the differential gain}$$

$$\text{PID Control : } u(t) = K_c e(t) + K_I \int e(t) dt + K_D \frac{d e(t)}{dt}$$

A well-tuned **PID controller** optimally balances these three actions, ensuring precise temperature control with minimal deviation and oscillation. Therefore, controller tuning is important.

Tuning of the controller:

Tuning a controller means adjusting its parameters (K_c , T_i , and T_d) to achieve the desired system performance in terms of stability, response time, and accuracy. Proper tuning ensures minimal overshoot, reduced steady-state error, and fast settling time. One of the widely used tuning methods is the Cohen -Coon tuning method.

Cohen-Coon Tuning Method

The Cohen-Coon method is a widely used empirical tuning technique for determining PID controller parameters based on the system's step response. It is particularly effective for first-order time delay systems. The values of the K_c , τ , τ_I , τ_D by Cohen - Coon method is given by:

	K_c	τ_I	τ_D
P	$\frac{1}{K} \frac{\tau}{t_d} \left(1 + \frac{t_d}{3\tau} \right)$	NA	NA
PI	$\frac{1}{K} \frac{\tau}{t_d} \left(0.9 + \frac{t_d}{12\tau} \right)$	$t_d \frac{30 + 3t_d/\tau}{9 + 20t_d/\tau}$	NA
PID	$\frac{1}{K} \frac{\tau}{t_d} \left(\frac{4}{3} + \frac{t_d}{4\tau} \right)$	$t_d \frac{32 + 6t_d/\tau}{13 + 8t_d/\tau}$	$t_d \frac{4}{11 + 2t_d/\tau}$



Figure 0: Schematics of the experiment

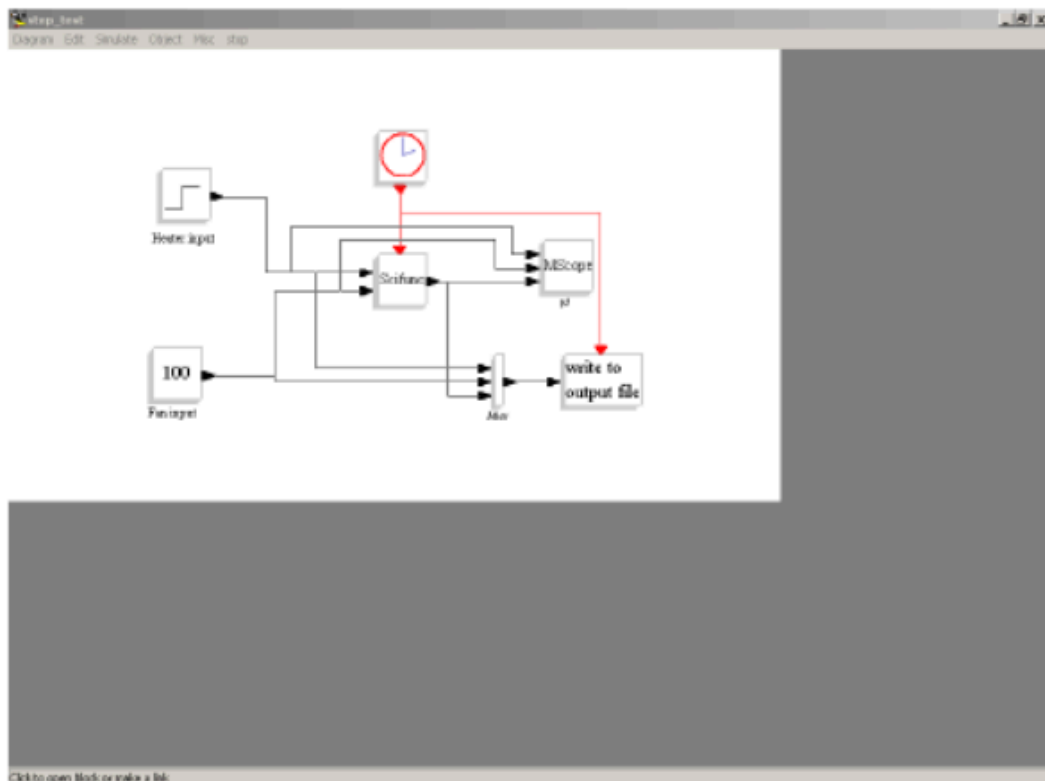


Figure 1: Figure showing the software used for the experiment.

Procedure:**Experiment 1:**

1. Connect the SBHS (Single Board Heater System) to an electrical point and to a PC having SCILAB.
2. Initialize the Serial port.
3. Open SCILAB, follow the instructions provided by TA and load the desired file on the software.
4. After loading the correct file, enter the fan speed and required heater input in the corresponding block. Name the output file.
5. Run the simulation.
6. You will see a graph appearing. Stop the simulation to give a new step input and then continue the simulation.
7. After the steady state has arrived save the graph and the file with data.

Experiment 2:

1. After getting the data from experiment 1, analyze the data to get the values of K (*gain*), τ , and t_d by their definition.
 - a. K (*gain*) = $\frac{\text{change in the output}}{\text{change in input}}$
 - b. τ (*time constant*) = *final output (after saturation)* * 0.632
 - c. t_d (*time delay*) = the period of time that elapses between the moment a control signal is applied and when a measurable change in the system's output is observed.
2. After getting the values of K (*gain*), τ , and t_d find the transfer function of the system.

Experiment 3:

1. After finding the transfer function, calculate the Cohen-Coon parameters (from Ziegler-Nichols Rule) of the controller.
2. Create a simulation in the simulink and tune the controllers.
3. After tuning the controller, check the tuning parameters by testing it on the SBHS.
4. Get the plot of output and error of the tuned circuit on SCILAB. Save the data for further analysis

Known parameters:**Experiment 1:**

- Fan speed
- Input step input

Experiment 2:

- The data obtained from experiment 1 and the system is of first order

Experiment 3:

- Transfer function of the system
- Cohen-Coon parameters
- Parameters of all the controller

Formulae:

1. $K (\text{gain}) = \frac{\text{change in the output}}{\text{change in input}} = 0.5667$
2. $\tau (\text{time constant}) = \text{final output (after saturation)} * 0.632 = 12.9 \text{ s}$
3. $t_d (\text{time delay}) = \text{the period of time that elapses between the moment a control signal is applied and when a measurable change in the system's output is observed.} = 2.5 \text{ s}$
4. $\text{Transfer function: } \frac{K_c}{\tau s + 1} = \frac{0.5667}{12.9 s + 1}$

Cohen Coon formulae:

	K_c	τ_I	τ_D
P	$\frac{1}{K} \frac{\tau}{t_d} \left(1 + \frac{t_d}{3\tau} \right)$	NA	NA
PI	$\frac{1}{K} \frac{\tau}{t_d} \left(0.9 + \frac{t_d}{12\tau} \right)$	$t_d \frac{30 + 3t_d/\tau}{9 + 20t_d/\tau}$	NA
PID	$\frac{1}{K} \frac{\tau}{t_d} \left(\frac{4}{3} + \frac{t_d}{4\tau} \right)$	$t_d \frac{32 + 6t_d/\tau}{13 + 8t_d/\tau}$	$t_d \frac{4}{11 + 2t_d/\tau}$

TABLE 1: TABLE CONTAINING COHEN COON PARAMETERS

	K_c	τ_I	τ_D
P	10.8706	NA	NA
PI	32.6674	5.9377	NA
PID (without tuning)	85.5574	5.6979	0.8934
PID (tuning)	85	50	0.89

In the tuning we gave P I and D Values and that is what is mentioned in PID (tuning).

Readings:

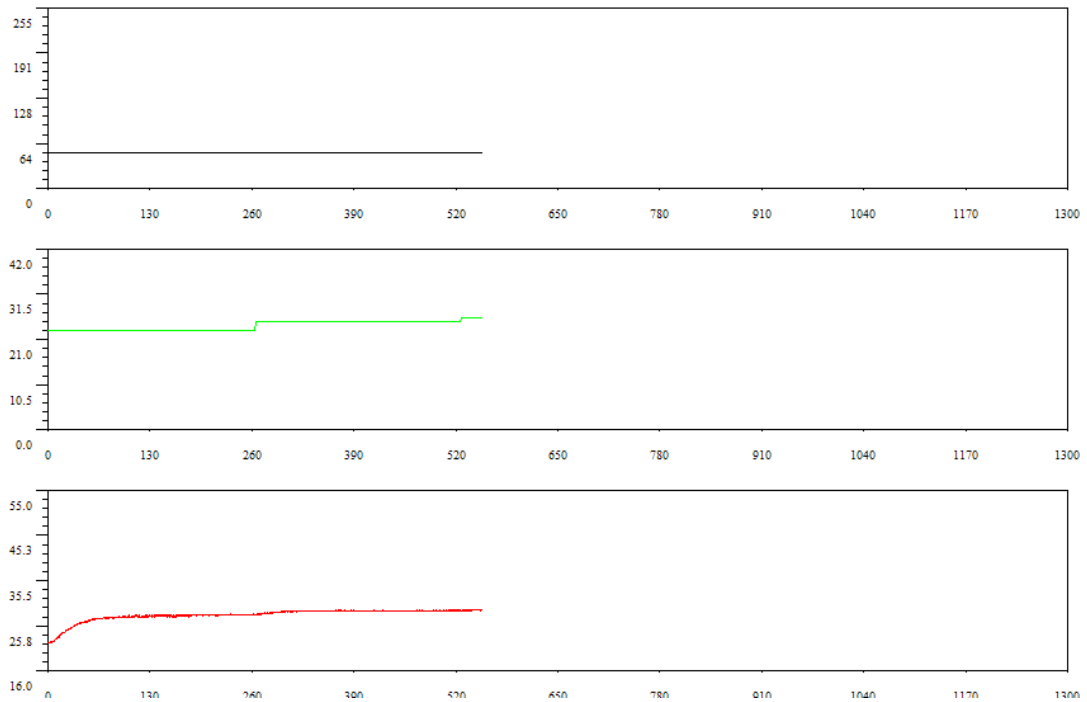


Figure 2: Figure showing the fan speed, step input and output data respectively

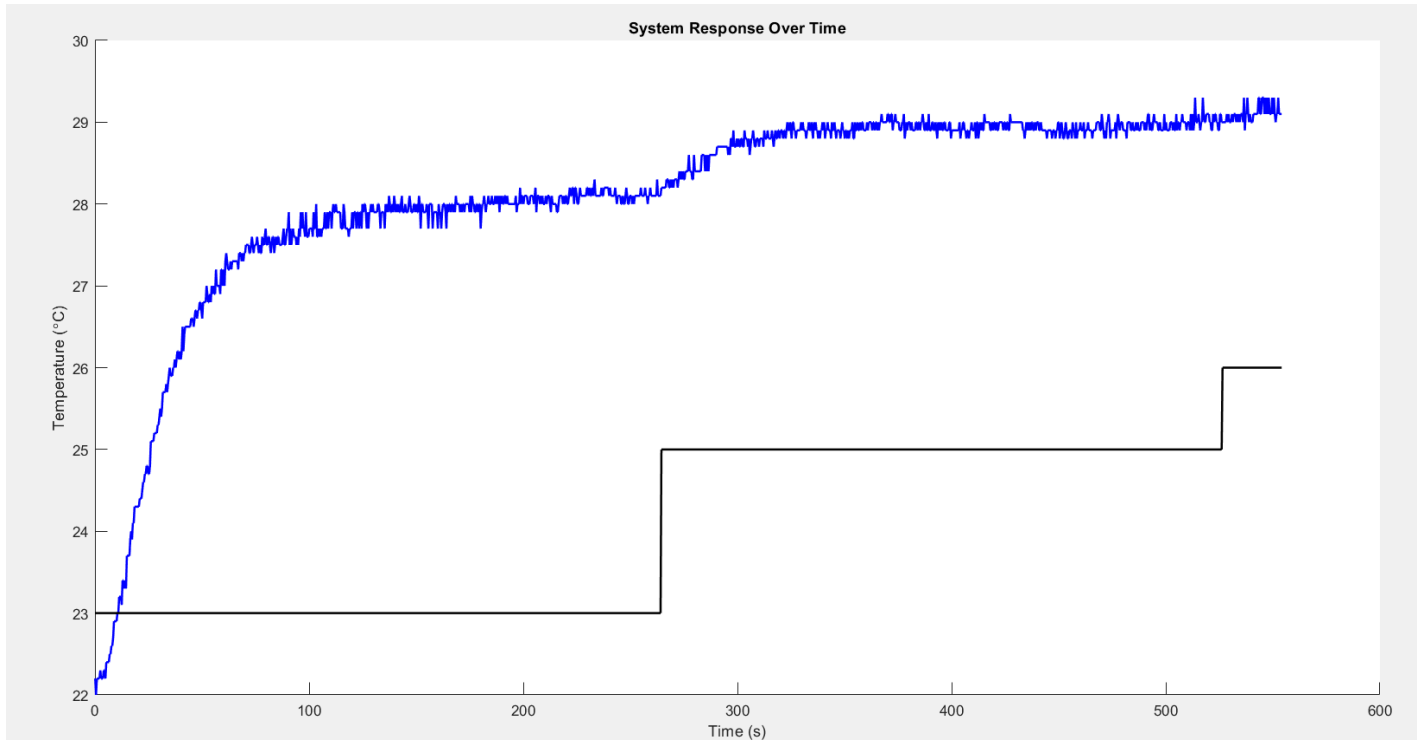


Figure 3: figure showing the input and output response of the system.

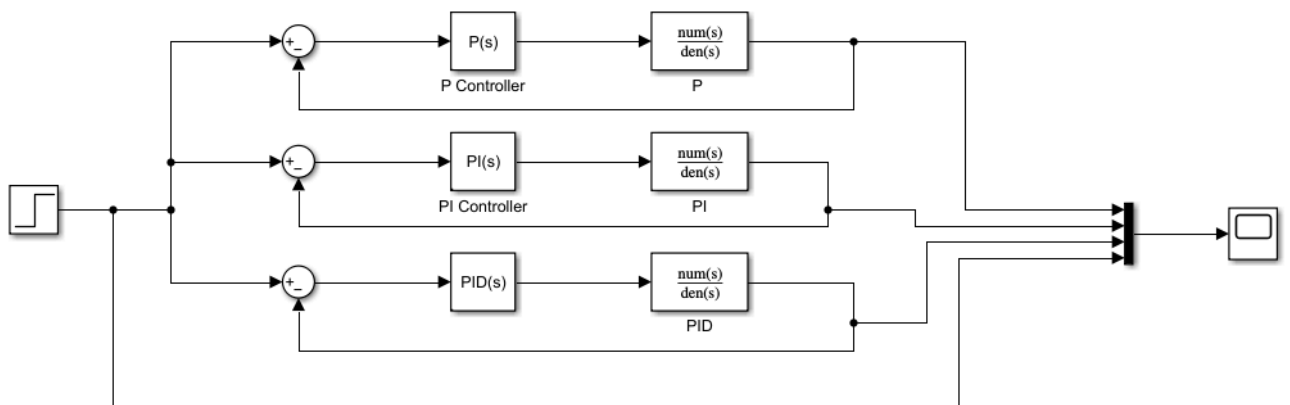


Figure 4: Figure showing the simulink system with the controller.

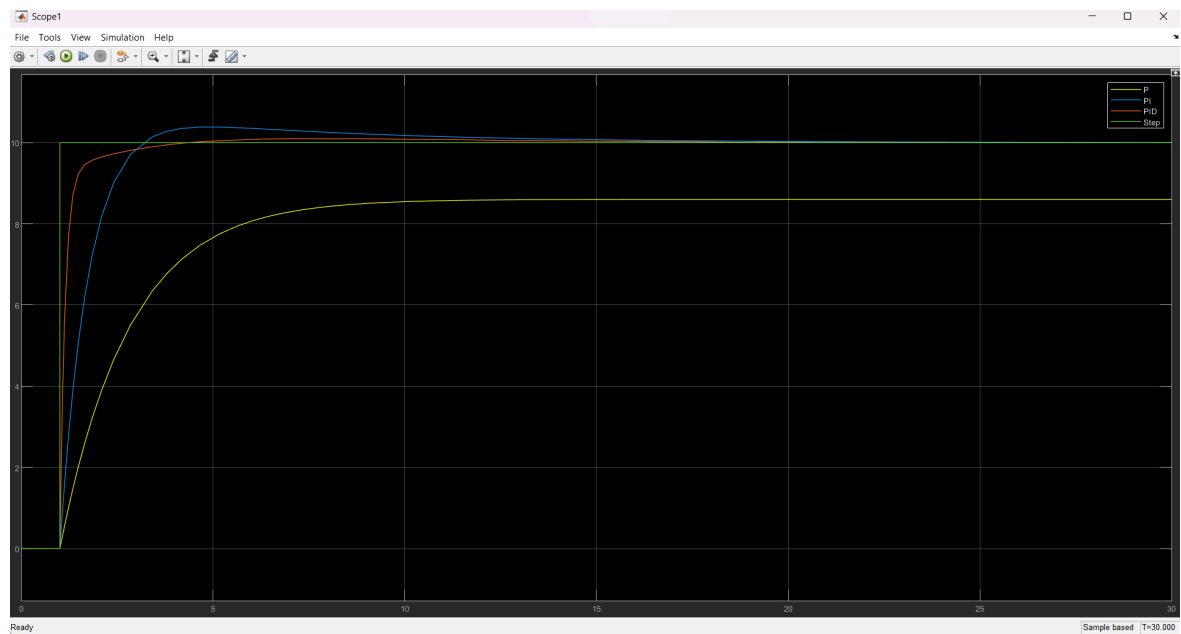


Figure 5: Figure showing the plot of circuit shown in figure 4.

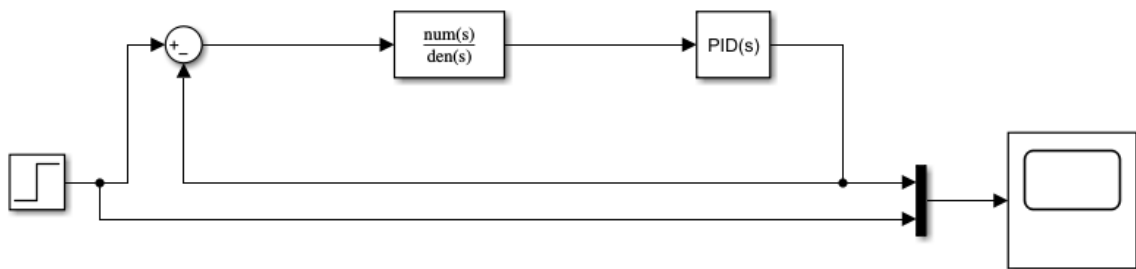


Figure 6: Figure showing the circuit (only PID controller) used for tuning



Figure 7: Figure showing the tuned PID controller output

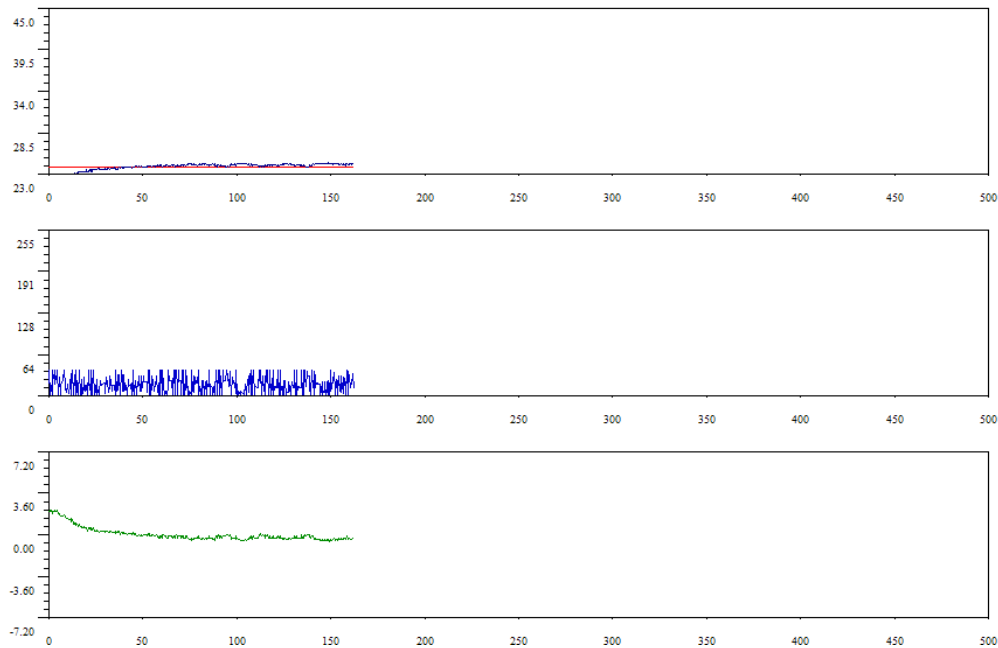


Figure 8: Figure showing the tuned PID controller output

Results and discussion:

To understand the behaviour of the system, we performed an initial experiment (experiment 1) by applying a step change in heater temperature while keeping the fan speed constant at 50. The thermocouple readings were recorded over time to track the system's response.

The plot generated had the temperature response of the thermocouple vs. time, along with the step change in input heater temperature. The corresponding Excel sheet contains all recorded values from the experiment. From the plot and the data, we saw that the system exhibits a characteristic lag between the input step change and the actual thermocouple reading. The data of the excel sheet was then used to calculate the transfer function of the system. This transfer function serves as the mathematical model for the system's behavior. After then we found out the Cohen-Coon parameters and tuned the parameters for a better and quick response with minimal error.

While calculating the transfer function and other parameters, we assumed that the system is a first order system.

Data we got for experiment 1 is shown in figure 3. Figure 3 shows that the system reached steady state at 113 seconds for 23°C step and remained in that state till 264s when the next step input of 25° C was given at 265th second. This system reached steady state at 311 second and remained in steady state till 526th second when another step input of 26°C was given. By 540th second the system stabilized.

The transfer function is in the form of $\frac{K_c}{\tau s + 1}$, K_c The proportional gain was calculated by noting the change in output and input temperatures of the system as detected by thermocouple.

$$K_c = \frac{\Delta Y}{\Delta X} = \frac{\text{Change in thermocouple outlet temperature}}{\text{change in thermocouple inlet temperature}}$$

$$\tau = 0.632 * \text{time taken to reach the saturation temperature}$$

Results of Experiment 1:

$$K_c = 0.5667, \tau = 12.9 \text{ s}, t_d = 2.5 \text{ s}$$

After getting the results of experiment 1, we used that to get the results of experiment 2.

Results of Experiment 2:

$$P: K_c = 10.8706$$

$$PI: K_c = 32.6674, \tau_I = 5.9377$$

$$PID: K_c = 85.5574, \tau_I = 5.6979, \tau_D = 0.8934$$

The simulink model of this is shown in figure 4.

The parameters used in PID blocks are modified.

$$P = K_c, I = \frac{P}{\tau_I}, D = P * \tau_D$$

The parameters used in block are:

$$P: P = 10.8706$$

$$PI: P = 32.6674, I = 5.5016$$

$$PID: P = 85.5574, \tau_I = 15.0156, \tau_D = 76.4369$$

After getting the block parameters, these parameters were tuned to get the least error and a stable output.

The average error of the tuning is **-0.22**. The PID was tuned at Parameters:

$$PID: P = 85, \tau_I = 50, \tau_D = 0.89$$

The graph in figure 8 shows the input heater temperature vs. output thermocouple temperature after applying PID control. The error graph (3rd one) demonstrates how the PID controller minimizes the deviation between setpoint and actual temperature over time.

This experiment provided valuable insights into the behavior of a temperature control system using different control strategies—P, PI, and PID controllers. By analyzing the step response data, deriving the system's transfer function, and applying the Cohen-Coon tuning method, we evaluated the effectiveness of each controller in regulating temperature.

In the first phase, the system's response to a step input was observed without any control mechanism. The recorded data revealed a noticeable lag due to thermal inertia, and key parameters such as steady-state gain and time constant were determined. This step established the baseline system behavior before implementing controllers.

When testing different controllers, the **Proportional (P) controller** showed a quick response but left a steady-state error, as it only reacted to the present error without addressing past or future trends. While it improved response speed, it could not fully eliminate long-term errors. The **Proportional-Integral (PI) controller** reduced steady-state error by integrating past deviations, enhancing accuracy but introducing overshoot and oscillations, which increased the settling time. The **Proportional-Integral-Derivative (PID) controller** combined the advantages of P and PI control while mitigating their limitations. The derivative term predicted future errors, reducing overshoot and ensuring a faster, more stable response with minimal oscillations. The error analysis confirmed that PID control significantly minimized deviations between the setpoint and the actual thermocouple temperature, achieving precise and stable temperature regulation.

After implementing the PID controller in Scilab, real-time system testing was conducted to evaluate performance. The tuned parameters (K_C , τ_I , τ_D) were further adjusted to optimize control, leading to reduced overshoot and oscillations, faster settling time, and minimal steady-state error. The final response plots demonstrated that the PID controller effectively minimized deviations, ensuring a stable temperature profile closely matching the desired setpoint.

Conclusion:

This experiment effectively demonstrated the role of P, PI, and PID controllers in temperature regulation using a heater-fan module. By analyzing the step response, deriving the transfer function, and applying Cohen-Coon tuning, we assessed how each control strategy impacted system stability, response time, and error reduction.

Findings of the experiment:

- The **P-controller** provided a fast response but left a steady-state error.
- The **PI-controller** eliminated steady-state error but introduced overshoot and oscillations.
- The **PID-controller** achieved the most balanced response, minimizing error and overshoot while efficiently reaching the desired setpoint.
- Although the system was treated as first-order, real-world behavior indicated higher-order dynamics, introducing unmodeled effects and potential sources of error.

Sources of error:

- We may have forgotten to reset the system.
- There can be air flow, change in the ambient temperature or heat loss to the surrounding.
- The system was assumed to be first-order, but real-world behavior suggests it is higher-order, leading to modeling inaccuracies.
- There can be a lag in the thermocouple.
- There can be noise measurements.

MATLAB Code:

```
clc;
clear all;
filename = 'Pass_data.xlsx';
data = readtable(filename, 'VariableNamingRule', 'preserve');
time = data.Time;
input_temp = data("Input");
output_temp = data("Output");
figure;
hold on;
plot(time, output_temp, 'b', 'LineWidth', 1.5);
plot(time, input_temp, 'k', 'LineWidth', 1.5);
title('System Response Over Time');
xlabel('Time (s)');
ylabel('Temperature (\circ C)');
hold off;
% steady-state gain (K)
delta_input = max(input_temp(260:1386)) - min(input_temp(260:1386)); % Change in input
delta_output = max(output_temp(260:1386)) - min(output_temp(260:1386)); % Change in output
K = delta_output / delta_input;
% Determine the delay time (t_d) and time constant (tau)
% Find the time when output first responds to the input (delay time)
threshold = output_temp(1) + 0.001 * delta_output; % Small response threshold
delay_index = find(output_temp > threshold, 1, 'first');
t_d = time(delay_index);
% time constant (tau)
steady_state_value = output_temp(end); % Final value
target_value = output_temp(1) + 0.693 * delta_output; % 69.3% of the change
tau_index = find(output_temp >= target_value, 1, 'first');
tau = time(tau_index);
% Cohen-Coon tuning parameters
% P Controller
Kc_P = (1 / K) * (tau / t_d) * (1 + t_d/tau);
% PI Controller
Kc_PI = (1 / K) * (tau / t_d) * (0.9 + t_d/12*tau);
tau_I_PI = t_d * (30 + 3*t_d/tau) / (9 + 20*t_d/tau);
% PID Controller
Kc_PID = (1 / K) * (tau / t_d) * (4/3 + t_d/4*tau);
tau_I_PID = t_d * (32 + 6*t_d/tau) / (13 + 8*t_d/tau);
```

```

tau_D_PID = t_d*4/(11+1*t_d/tau);
% Display results
fprintf('Steady-State Gain (K): %.4f\n', K);
fprintf('Delay Time (t_d): %.4f s\n', t_d);
fprintf('Time Constant (tau): %.4f s\n\n', tau);
fprintf('Cohen-Coon Parameters:\n');
fprintf('P Controller: Kc = %.4f\n', Kc_P);
fprintf('PI Controller: Kc = %.4f, Tau_I = %.4f s\n', Kc_PI, tau_I_PI);
fprintf('PID Controller: Kc = %.4f, Tau_I = %.4f s, Tau_D = %.4f s\n', ...
    Kc_PID, tau_I_PID, tau_D_PID);
disp(Kc_PID/tau_I_PID);
disp(Kc_PID*tau_D_PID);

```

Output:

Steady-State Gain (K): 0.5667

Delay Time (t_d): 2.5000 s

Time Constant (tau): 12.9000 s

Cohen-Coon Parameters:

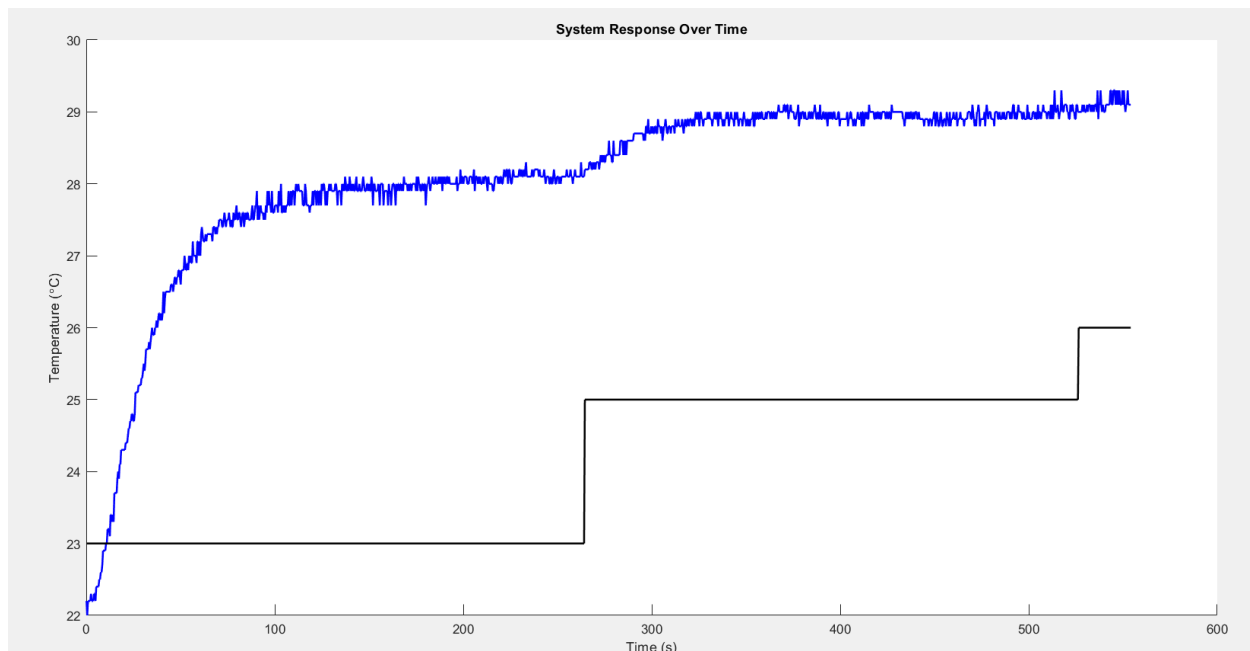
P Controller: Kc = 10.8706

PI Controller: Kc = 32.6674, Tau_I = 5.9377 s

PID Controller: Kc = 85.5574, Tau_I = 5.6979 s, Tau_D = 0.8934 s

15.0155

76.4328



References:

- [1] “iPC Scholar,” *Wileyindia.com*, 2025. [link](#)
- [2] “What Is PID Controller, Working, Advantages, Disadvantages, Applications,” *cselectricalandelectronics.com*. [PID Controller](#)
- [3] “How to Tune a PID Controller | Omega,” *www.omega.co.uk*. [PID Tuning](#)
- [4] M. Anderson, “PID Tuning | How to Tune a PID Controller - RealPars,” *www.realpars.com*, Jan. 11, 2021. <https://www.realpars.com/blog/pid-tuning>
- [5] G. McMillan, “How to Avoid Common Tuning Mistakes With PID Controllers,” *blog.isa.org*. <https://blog.isa.org/avoid-common-tuning-mistakes-pid>
- [6] J. Smuts, “Cohen-Coon Tuning Rules | Control Notes,” *blog.opticontrols.com*, Mar. 24, 2011. <https://blog.opticontrols.com/archives/383>
- [7] ‘Lab Manual’ provided in the classroom.